

IoT Protocol Stack Simulator (Python)

Group Members: Karthikeya Bezwada (24701183), Samuel Ou (24220908)

1. Overview

`iot_sim.py` is a single-file IoT network simulator that demonstrates a complete low-power protocol stack: **IEEE 802.15.4 MAC → IPv6 → RPL → UDP → CoAP → IPsec ESP → DTLS**. It shows end-to-end communication between an IoT node and a CoAP server, including encapsulation, decapsulation, routing, encryption, integrity checking, and replay protection.

The topology is fixed: five nodes A–E (A is the RPL root and Internet gateway) plus a CoAP server attached to A via a wired interface. The encapsulation order on the sender side is:

CoAP → DTLS → UDP → IPsec ESP → IPv6 → MAC

and the receiver reverses this, performing decapsulation and integrity checks at each layer.

2. File Organisation

Section	Purpose
Header docstring	Describes the file and which parts it implements.
Constants block	MAC/IPv6/UDP/CoAP/ESP/DTLS field formats, frame types, next-header values, ports, and keys.
Helpers	<code>mac_to_bytes</code> , <code>ip_to_bytes</code> , <code>bytes_to_mac</code> , <code>bytes_to_ip</code> , <code>fmt_rank</code> , <code>log</code> .
<code>Channel</code> class	Idealised wireless medium: one-hop unicast and broadcast delivery.
<code>Node</code> class	Full protocol stack for nodes A–E.
<code>CoAPServer</code> class	Internet-side server (IPv6/UDP/CoAP/IPsec/DTLS).
Topology builder	<code>build_network()</code> , <code>print_topology()</code> .
<code>main()</code>	Entry point: <code>init</code> → RPL → prompt for part (C/D) → prompt for source node.

Constants live at the top, helpers next, then the three classes, then the driver — a conventional, readable structure.

3. Main Components

3.1 log()

Produces uniform lines `[Node X] [PROTOCOL] [OP] message`, satisfying the rubric's logging requirement. The server uses the same format.

3.2 Channel

Models an idealised wireless medium (no collisions/CSMA/CA). Delivers frames to one-hop neighbours and queues them so each node finishes processing before the next frame is delivered.

3.3 Node

Stores identity (`name`, `mac`, `ipv6`, `neighbors`), MAC state (`mac_seq`, `pending_acks`), RPL state (`rank`, `preferred_parent`, `parent_mac`), and security state (`ipsec_seq`, `dtls_seq`, `replay windows`, `keys`).

Each layer is a symmetric send/receive pair:

Layer	Send	Receive
MAC	<code>send_mac()</code>	<code>receive_mac()</code>
IPv6	<code>send_ipv6()</code>	<code>receive_ipv6()</code>
RPL	<code>send_rpl()</code>	<code>receive_rpl()</code>
IPsec ESP	<code>send_ipsec()</code>	<code>receive_ipsec()</code>
UDP	<code>send_udp()</code>	<code>receive_udp()</code>
DTLS	<code>send_dtls()</code>	<code>receive_dtls()</code>
CoAP	<code>send_coap()</code>	<code>receive_coap_response()</code>

Key behaviours:

- MAC: 12-byte header, ACK for unicast DATA, no ACK for broadcast.
- IPv6: 35-byte header, next-header dispatch (58 ICMPv6, 17 UDP, 50 ESP, 59 none), forwarding at intermediate nodes with unchanged source/destination IPv6.
- RPL: DIO broadcasts (Type=155, Code=1, Rank), candidate rank = advertised + 1, preferred-parent selection, re-broadcast on update.
- IPsec ESP: Transport Mode with SPI, sequence number, IV, AES-128-CBC, HMAC-SHA-256, replay window.
- DTLS: Record with Type/Version/Epoch/Seq/Length, AES-128-CBC, HMAC-SHA-256, separate keys and replay window.
- UDP: 8-byte header; forwards to DTLS when security is active, otherwise to CoAP.
- CoAP: 4-byte fixed header + Token + Uri-Path `/temperature` + payload; server replies with piggybacked ACK 2.04 Changed.

Two routing helpers: `find_downward_next_hop()` walks up from the destination to find the next hop downward; the upward path uses `self.parent_mac`.

3.4 CoAPServer

Standalone object reachable only through A's wired interface. Implements matching receive/build functions for IPv6, IPsec, UDP, DTLS, and CoAP, and maintains its own sequence counters and replay windows.

4. CoAP Message ID and Token

- **Message ID = 1001.** Operates at the CoAP *message layer* for duplicate detection and matching ACK/RST to CON. The server copies it into the piggybacked ACK.
- **Token = 0xA1B2.** Operates at the *request/response layer*. Echoed unchanged by the server so the client can match responses to requests independently of the message layer.
- **Piggybacked ACK.** The server sends a single ACK carrying 2.04 Changed, the same Message ID, and the same Token — satisfying both reliability and matching in one message, as described in Lecture 6.

5. Packet Flow

Upward (D → Server):

CoAP → DTLS → UDP → IPsec ESP → IPv6 (NH=50) → MAC (D→B) → MAC (B→A) → wired → Server

Downward (Server → D):

Server → wired → A → B → D

IPv6 source/destination stay unchanged across hops; MAC addresses are rewritten hop-by-hop.

6. Task Allocation

Area	24220908	24701183
Part A	Allocated	—
Part B	Allocated	—
Part C	—	Allocated
Part D	—	Allocated
main()	Both	Both
Documentation, testing & comments	Both	Both